Category C Problems

5 Problems · steps 1–4 of the walkthrough loop · repeat the core pattern across problems so it sticks · OOP, Classes and Objects — Week 3 · Homework Assignment

L1. Simple Book Record Class

Scenario

The campus library wants the simplest possible way to store one book's title and price together, instead of two separate, unrelated variables.

Task

- Define a class Book with two fields: String title and double price.
- In main, create one Book object and set both fields directly.
- Print a single line in the form “Title: ... | Price: Rs ...”.
- No constructor or extra methods needed for this one — just a class, an object, and field access.

Suggested method signature(s)

```
class Book { String title; double price; }
```

Sample Input / Output

Input	Output
title = "Clean Code" *price = 650.0*	*Title: Clean Code | Price: Rs 650.0*

Concepts covered: *Class definition, object creation with new, direct field access.*

L2. Two Objects, Two Separate Occupancy Counts

Scenario

An exam-hall seating app tracks how many seats are filled in one hall at a time. The department wants to confirm on paper that two hall objects never interfere with each other's numbers.

Task

- Define a class ExamHall with fields String hallName and int seatsFilled.
- Create two ExamHall objects, hallA and hallB, with different hallName values.
- Increase seatsFilled for hallA only, four separate times.
- Print both halls' seatsFilled values to show hallB was never touched.

Suggested method signature(s)

```
class ExamHall { String hallName; int seatsFilled; }
```

Sample Input / Output

Input	Output
hallA = "Block-3 Hall A" *hallB = "Block-3 Hall B"* *seatsFilled on hallA incremented 4 times*	*Block-3 Hall A seatsFilled: 4* *Block-3 Hall B seatsFilled: 0*

Concepts covered: *Object creation, independent instance state, field access.*

L3. Reading Java's Default Field Values

Scenario

A new intern insists an unset boolean field starts out as true by default. Settle the debate by printing an object's fields immediately after it's created.

Task

- Define a class Employee with fields String empName, double salary, and boolean permanent.
- Create one Employee object using new, without setting a single field.
- Print all three fields right away, before assigning anything to them.

Suggested method signature(s)

```
class Employee { String empName; double salary; boolean permanent; }
```

Sample Input / Output

Input	Output
(no fields set after creation)	*Name: null* *Salary: 0.0* *Permanent: false*

Concepts covered: *Java's default field values, object creation, print debugging.*

L4. Basic Constructor for a Product

Scenario

Typing two separate assignment lines for every new Product object is slowing the inventory team down. Fix it with the simplest possible constructor.

Task

- Define a class Product with fields String productId and String productName.
- Write a constructor that takes both values as parameters and assigns them using this.productId = productId; and this.productName = productName;.
- Create one Product object through the constructor and print its two fields on one line.

Suggested method signature(s)

```
public Product(String productId, String productName)
```

Sample Input / Output

Input	Output
productId = "P-1042" *productName = "Wireless Mouse"*	*P-1042 - Wireless Mouse*

Concepts covered: *Constructor basics, this.field = parameter, object creation.*

L5. Counting Objects With a Static Field

Scenario

The order-processing desk wants a running count of how many Order objects have been created so far today — one shared number, not something each order tracks separately.

Task

- Define a class Order with a static int totalOrders field, starting at 0.
- In the constructor, increment totalOrders by 1 every single time.
- Create four Order objects.
- Print Order.totalOrders exactly once, accessed through the class name.

Suggested method signature(s)

```
static int totalOrders;
```

Sample Input / Output

Input	Output
4 Order objects created	*Total orders: 4*

Concepts covered: *Static fields, constructors, accessing static members through the class name.*